Step 3 — Create ZCL_PO_TEXT_BADI_IMPL

SE19 BAdI Implementation — Dual Write on Every PO Save

BAdI: ME_PROCESS_PO_CUST | Enhancement Spot: ME_PURCHORD | 2026-06-28

What This BAdI Does

The BAdI ME_PROCESS_PO_CUST is a SAP hook that fires automatically AFTER a Purchase Order is saved in ME21N or ME22N — after SAP standard SAVE_TEXT has already written the text to STXH/STXL. At that point the text is readable via READ_TEXT. The BAdI calls ZCL_PO_LONGTEXT_HANDLER to write the same text as plain rows into ZPO_LONGTEXT automatically. No manual migration needed for new POs after this is active.

```
User saves PO in ME21N / ME22N

        ■

        ▼

SAP Standard (always, unchanged)

  SAVE_TEXT → STXH (text header) + STXL (compressed content)

        ■

        ▼  ■■■ BAdI fires HERE after SAVE_TEXT completes

ZCL_PO_TEXT_BADI_IMPL (this implementation)

  PROCESS_HEADER → calls handler for all EKKO texts

  PROCESS_ITEM   → calls handler for all EKPO texts per item

        ■

        ▼

ZCL_PO_LONGTEXT_HANDLER=>WRITE_TO_PERSISTENCE

  → Reads STXL via READ_TEXT (decompresses)

  → Deletes old ZPO_LONGTEXT rows for this text

  → Inserts fresh plain rows → SOURCE = D

        ■

        ▼

ZPO_LONGTEXT now has plain readable rows ✓

SELECT * FROM ZPO_LONGTEXT WHERE EBELN = ... → no READ_TEXT needed
```

Step	Object	Transaction	Purpose
1	Create BAdI implementation	SE19	Register ZCL_PO_TEXT_BADI_IMPL for ME_PROCESS_PO_CUST
2	Implement PROCESS_HEADER	SE24 via SE19	Writes all EKKO texts for the saved PO
3	Implement PROCESS_ITEM	SE24 via SE19	Writes all EKPO texts for each saved item
4	Activate	SE19 / SE24	BAdI goes live — fires on every ME21N/ME22N save
5	Test	ME21N + SE16N	Create new PO → verify ZPO_LONGTEXT populated

A Open SE19 and Create the BAdI Implementation

Transaction SE19 — Business Add-In Builder

1 Open SE19

Transaction code: SE19 → press Enter

2 Select Classic BAdI

Screen shows two options: Classic BAdI and New BAdI. Select: Classic BAdI (radio button) Enter BAdI Name: ME_PROCESS_PO_CUST Click Create Implementation button

3 Enter Implementation Name

Implementation Name: ZCL_PO_TEXT_BADI_IMPL Click Continue (green tick or Enter)

4 Fill Implementation Properties

Short Text: PO Long Text Dual Write — ZPO_LONGTEXT The interface IF_EX_ME_PROCESS_PO_CUST is automatically assigned Click Save

5 Assign package and transport

Package: ZMM (or your development package) Transport: assign to your active workbench transport Save

After creating the implementation, SE19 opens the class ZCL_PO_TEXT_BADI_IMPL in SE24 automatically. You will see the Methods tab with PROCESS_HEADER and PROCESS_ITEM already listed — these come from the interface IF_EX_ME_PROCESS_PO_CUST.

B Implement PROCESS_HEADER Method

Fires when PO header is saved — writes all EKKO texts to ZPO_LONGTEXT

1 **Go to Methods tab**

In SE24 for ZCL_PO_TEXT_BADI_IMPL → click Methods tab

2 **Open PROCESS_HEADER source**

Double-click IF_EX_ME_PROCESS_PO_CUST~PROCESS_HEADER Source code editor opens showing the empty METHOD...ENDMETHOD skeleton

3 **Replace with full code below**

Select all existing content → Delete → Paste the code:

```
METHOD if_ex_me_process_po_cust~process_header.


  " Get PO number from header object
  DATA(lv_ebeln) = im_header->get_data( )-ebeln.


  " PO number is initial when PO not yet committed — exit safely
  IF lv_ebeln IS INITIAL.
    RETURN.
  ENDIF.


  " Read ALL header text entries for this PO from STXH dynamically
  " This handles F01, F02, F03 etc. — no hardcoding of TDID needed
  DATA lt_stxh TYPE TABLE OF stxh.
  DATA lv_tdname TYPE thead-tdname.
  lv_tdname = lv_ebeln.


  SELECT tdobject, tdid, tdspras, tdname
    FROM stxh
    INTO CORRESPONDING FIELDS OF TABLE @lt_stxh
    WHERE tdobject = 'EKKO'
      AND tdname   = @lv_tdname.


  " Write each text type to ZPO_LONGTEXT
  LOOP AT lt_stxh INTO DATA(ls_stxh).
```

```
    zcl_po_longtext_handler=>write_to_persistence(

      iv_ebeln    = lv_ebeln

      iv_ebelp    = '00000'

      iv_tdobject = ls_stxh-tdobject

      iv_tdid     = ls_stxh-tdid

      iv_tdspras  = ls_stxh-tdspras ).

  ENDLOOP.


ENDMETHOD.
```

4 **Save**

Ctrl+S

C — Implement PROCESS_ITEM Method

Fires per item when PO is saved — writes all EKPO texts to ZPO_LONGTEXT

1 **Open PROCESS_ITEM source**

Methods tab → double-click IF_EX_ME_PROCESS_PO_CUST~PROCESS_ITEM Source code editor opens

2 **Replace with full code below**

Select all → Delete → Paste:

```
METHOD if_ex_me_process_po_cust~process_item.


  " Get PO number and item number
  DATA(lv_ebeln) = im_item->get_data( )-ebeln.
  DATA(lv_ebelp) = im_item->get_data( )-ebelp.


  IF lv_ebeln IS INITIAL.
    RETURN.
  ENDIF.


  " Build TDNAME for this item — NO SPACE (confirmed for this system)
  " EKPO TDNAME = EBELN + EBELP without space: 450007774400010
  DATA lv_tdname TYPE thead-tdname.
  CONCATENATE lv_ebeln lv_ebelp INTO lv_tdname.


  " Read ALL item text entries for this PO item from STXH dynamically
  " Picks up F01, F02, F09 etc. — all TDIDs for this item
  DATA lt_stxh TYPE TABLE OF stxh.


  SELECT tdobject, tdid, tdspras, tdname
    FROM stxh
    INTO CORRESPONDING FIELDS OF TABLE @lt_stxh
    WHERE tdobject = 'EKPO'
      AND tdname   = @lv_tdname.


  " Write each text type to ZPO_LONGTEXT
```

```
  LOOP AT lt_stxh INTO DATA(ls_stxh).

    zcl_po_longtext_handler=>write_to_persistence(

      iv_ebeln    = lv_ebeln

      iv_ebelp    = lv_ebelp

      iv_tdobject = ls_stxh-tdobject

      iv_tdid     = ls_stxh-tdid

      iv_tdspras  = ls_stxh-tdspras ).

  ENDLOOP.


ENDMETHOD.
```

3 **Save**

Ctrl+S

Why PROCESS_ITEM reads STXH dynamically: Each item may have different text types (F01, F02, F09 etc.). Reading from STXH ensures ALL text types for that item are migrated. No hardcoding of TDID needed — future text types are handled automatically.

D Activate the Class and BAdI Implementation

Ctrl+F3 in SE24 → then verify in SE19

1 **Activate the class in SE24**

Press Ctrl+F3 (Activate) Status bar must show: Object ZCL_PO_TEXT_BADI_IMPL activated Both method names turn BLUE in Methods tab

2 **Verify in SE19**

Go back to SE19 → Classic BAdI → Implementation → ZCL_PO_TEXT_BADI_IMPL Status must show: Active (green indicator) If grey/inactive → click Activate button in SE19 toolbar

3 **Confirm BAdI is registered**

SE19 → Classic BAdI → BAdI Definition → ME_PROCESS_PO_CUST Click 'Display' → Implementation tab ZCL_PO_TEXT_BADI_IMPL must appear in the list with Active status

■ BAdI is active when: SE19 → ZCL_PO_TEXT_BADI_IMPL → Status = Active SE24 → ZCL_PO_TEXT_BADI_IMPL → both methods are blue (active) ME_PROCESS_PO_CUST implementation list shows ZCL_PO_TEXT_BADI_IMPL

E

Test the BAdI — Create New PO and Verify

ME21N → save with text → SE16N → ZPO_LONGTEXT must have rows

1 **Create a new PO in ME21N**

Transaction: ME21N → Enter Fill: Vendor, Purchase Org 0001, Company Code 0001, Document Type NB

2 **Add Header text**

Header → Texts → Text Overview Type in first field next to Header text: Line 1: BAdI test header note line 1 Line 2: BAdI test header note line 2 Press F3 to return

3 **Add line item**

Items section: add Material, Quantity 1, Delivery Date, Plant 0001

4 **Add Item text**

Select Item 10 → Item → Texts → Text Overview Type in first field next to Item text: Line 1: BAdI test item note line 1 Press F3 to return

5 **Save the PO**

Ctrl+S → wait for status bar message: Standard PO xxxxxxxxxx saved Note the new PO number

6 **Verify STXH**

SE16 → STXH → Text object: EKKO, Text Name: , Language: D Expect: 1 row confirming header text was saved by SAP standard

7 **Verify ZPO_LONGTEXT**

SE16N → ZPO_LONGTEXT → EBELN: → Execute Expect rows: EKKO / F01 / 00000 / LINE_COUNTER 00001 / TDLINE: BAdI test header note line 1 / SOURCE: D EKKO / F01 / 00000 / LINE_COUNTER 00002 / TDLINE: BAdI test header note line 2 / SOURCE: D EKPO / F01 / 00010 / LINE_COUNTER 00001 / TDLINE: BAdI test item note line 1 / SOURCE: D

■ BAdI is working correctly when: ZPO_LONGTEXT rows appear for the NEW PO immediately after save SOURCE = D (Dual-write by BAdI — not M for migration) All text types (header + items) are present No manual migration program run needed

F Troubleshooting — If ZPO_LONGTEXT Has No Rows After Save

Debug checklist — check in this order

#	Check	How	Fix
1	BAdI is Active	SE19 → ZCL_PO_TEXT_BADI_IMPL → Status	Must be Active (green). If grey → activate in SE19.
2	Class is Active	SE24 → ZCL_PO_TEXT_BADI_IMPL → status bar	Methods must be blue. If inactive → Ctrl+F3.
3	PO was saved (not held)	Status bar after save shows 'saved' not 'held'	Edit → Release Hold → Ctrl+S again.
4	Text exists in STXH	SE16 → STXH → Text object=EKKO, Text Name=	If no STXH row → text was not saved. Re-enter text in ME22N.
5	ZPO_LONGTEXT table exists	SE11 → ZPO_LONGTEXT → Display → Active	Must be Active. If not → complete Step 1 guide.
6	Handler class is Active	SE24 → ZCL_PO_LONGTEXT_HANDLER → methods blue	Must be Active. If not → Ctrl+F3 in SE24.
7	TDNAME no-space confirmed	SE16 → STXH → check EKPO TDNAME column	Must be 450007774400010 format (no space). Handler uses CONCATENATE without space.

How to Add a Breakpoint for Debugging

If BAdI is active but ZPO_LONGTEXT still has no rows: SE24 → ZCL_PO_TEXT_BADI_IMPL → PROCESS_HEADER method Set a breakpoint on the first line (click left margin — red dot appears) Run ME21N → save PO → debugger opens at breakpoint Check: lv_ebeln has a value (not initial) Check: lt_stxh is populated after the SELECT Step through LOOP → check ZCL_PO_LONGTEXT_HANDLER call If lt_stxh is empty → text not in STXH yet at this point (timing issue)

All 4 Steps — Complete Summary

Step	Object	Status	Result
Step 1	ZPO_LONGTEXT (SE11)	■ Complete	Custom table — stores plain text lines. Active in SE11.
Step 2	ZCL_PO_LONGTEXT_HANDLER (SE24)	■ Complete	Handler class — READ_TEXT → writes to ZPO_LONGTEXT. Active.
Step 3	ZCL_PO_TEXT_BADI_IMPL (SE19)	→ This guide	BAdI — fires on ME21N/ME22N save → calls handler automatically.
Step 4	Test ME21N + SE16N	After Step 3	New PO with text → ZPO_LONGTEXT populated with SOURCE=D.

After all 4 steps are complete: ✓ Historic POs: run ZTEST_PO_LONGTEXT_MIGRATE to populate ZPO_LONGTEXT ✓ New POs: BAdI automatically writes to ZPO_LONGTEXT on every save ✓ Reports / BDC: SELECT * FROM ZPO_LONGTEXT WHERE EBELN = ... — no READ_TEXT needed